

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

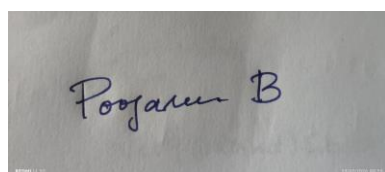
Weightage: 40%

QUESTIONS:

Total Marks: 30

Code of Conduct:

I POOJASREE B(192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a piece of paper with the handwritten signature 'Poojasree B' in blue ink. The signature is written in a cursive style. The paper is slightly wrinkled and has some faint markings at the bottom.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

1. SOAP-Based Web Service for Hospital Management System Design

A hospital can implement a SOAP web service that allows different departments such as reception, pharmacy, laboratory, and doctors to securely exchange patient information.

Main operations:

- `addPatient()` – registers a new patient.
- `getPatient()` – retrieves patient details.
- `scheduleAppointment()` – schedules an appointment.
- `getMedicalRecords()` – retrieves medical history.
- `updateMedicalRecord()` – updates a patient's medical information.

SOAP Message Structure

A SOAP message consists of three main parts:

1. Envelope

- Root element of every SOAP message.
- Identifies the message as a SOAP message.

2. Header

- Contains optional information such as authentication, authorization, transaction ID, and security tokens.
- Can be used with WS-Security.

3. Body

- Contains the actual request or response.
- Includes the operation name and required patient data.

Example Request

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                xmlns:hos="http://hospital.example.com/">
  <soap:Header>
    <hos:Authentication>
      <hos:Token>ABC123</hos:Token>
    </hos:Authentication>
  </soap:Header>
  <soap:Body>
    <hos:registerPatient>
      <hos:patient>
        <hos:firstName>John</hos:firstName>
        <hos:lastName>Doe</hos:lastName>
        <hos:email>john.doe@example.com</hos:email>
        <hos:password>123456</hos:password>
      </hos:patient>
    </hos:registerPatient>
  </soap:Body>
</soap:Envelope>
```

```
        </hos:Authentication>
    </soap:Header>

    <soap:Body>
        <hos:addPatient>
            <hos:patientId>P101</hos:patientId>
            <hos:name>John</hos:name>
            <hos:age>35</hos:age>
            <hos:department>Cardiology</hos:department>
        </hos:addPatient>
    </soap:Body>
</soap:Envelope>
```

Data Exchange

1. The client creates a SOAP request.
2. The request is sent to the hospital service endpoint using HTTP/HTTPS.
3. The SOAP server validates the message and authentication information.
4. The requested operation is executed.
5. The server returns a SOAP response containing the result or a SOAP Fault if an error occurs.

Reliability and Security

SOAP is suitable for hospital systems because it provides a **standardized and structured communication format**. It supports **WS-Security**, encryption, authentication, digital signatures, and reliable messaging mechanisms. HTTPS can additionally protect data during transmission. This is important because patient and medical information is highly confidential.

2. WSDL Structure for an Enterprise Web Service

WSDL describes **what a web service does, how it can be called, and where it is available**. It allows external applications to understand and consume the service without knowing its internal implementation.

Main WSDL Components

1. Types

- Defines the data types used by the service.
- Usually uses XML Schema (XSD).

2. Message

- Defines the input and output messages exchanged between client and server.

3. PortType

- Defines the operations supported by the service.
- Acts like the service interface.

4. Binding

- Specifies how the operations are communicated.
- Defines protocols such as SOAP over HTTP.

5. Service

- Specifies the actual endpoint/address where the service can be accessed.

Example WSDL Structure

```
<definitions
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  targetNamespace="http://example.com/data">

  <types>
    <schema xmlns="http://www.w3.org/2001/XMLSchema">
      <element name="getDataRequest">
        <complexType>
          <sequence>
            <element name="id" type="string"/>
          </sequence>
        </complexType>
      </element>
```

```
        </schema>
    </types>

    <message name="GetDataRequest">
        <part name="parameters" element="getDataRequest"/>
    </message>

    <message name="GetDataResponse">
        <part name="result" type="string"/>
    </message>

    <portType name="DataPortType">
        <operation name="getData">
            <input message="GetDataRequest"/>
            <output message="GetDataResponse"/>
        </operation>

        <operation name="updateData">
            <input message="UpdateDataRequest"/>
            <output message="UpdateDataResponse"/>
        </operation>
    </portType>

    <binding name="DataBinding" type="DataPortType">
        <soap:binding
            transport="http://schemas.xmlsoap.org/soap/http"/>
    </binding>

    <service name="DataService">
        <port name="DataPort"
            binding="DataBinding">
            <soap:address
                location="https://example.com/data"/>
        </port>
    </service>

</definitions>
```

How It Works

Client → WSDL → Service Interface → Binding → Endpoint → Server

The client reads the WSDL to determine:

- Available operations.
- Required input and output data.
- Communication protocol.
- Service endpoint.

Importance

WSDL provides a **machine-readable contract** between service providers and consumers. It improves interoperability because different applications and programming languages can communicate using the same service definition. It also simplifies integration, development, testing, and maintenance of enterprise web services.

3. Debugging SOAP Communication Problems

When SOAP requests are not processed correctly, the complete communication path should be checked systematically.

Possible Problems

1. **Incorrect SOAP message format**
2. **Incorrect namespaces**
3. **Incorrect operation**
4. **Endpoint problems**
5. **Data-type errors**
6. **Authentication/security problems**

Debugging Approach

Step 1 – Check the WSDL

Verify the operation name, input/output messages, namespaces, binding, and endpoint.

Step 2 – Validate the SOAP Request

Check that the XML is well-formed and follows the expected SOAP structure.

Step 3 – Verify Namespaces

Compare the namespaces in the request with those defined in the WSDL.

Step 4 – Check the Endpoint

Confirm that the URL, port, protocol, and server status are correct.

Step 5 – Test Using a SOAP Client

Tools such as SOAP testing clients can be used to send requests and inspect responses.

Step 6 – Analyze SOAP Faults

A SOAP Fault provides information about errors. For example:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid patient ID</faultstring>
</soap:Fault>
```

Step 7 – Check Server Logs

Server logs can identify authentication failures, database errors, timeouts, and processing exceptions.

Ensuring Reliable Communication

After correcting the errors, the client and server should use the **same WSDL definitions, namespaces, message formats, and endpoint configuration**. Proper SOAP Fault handling should be implemented for errors, while logging and timeout/retry mechanisms can help diagnose and recover from communication failures.

Thus, systematic validation of the SOAP message, WSDL, endpoint, security information, and server logs ensures **reliable and accurate communication** between enterprise systems.